Chapter 17: Mixture models

Shravan Vasishth

2026-04-02

Contents

1 *Textbook*

Introduction to Bayesian Data Analysis for Cognitive Science

Nicenboim, Schad, Vasishth

- Online version: `https://bruno.nicenboim.me/bayescogsci/`
- Source code: `https://github.com/bnicenboim/bayescogsci`
- Physical book: here

Be sure to read the textbook's chapter 17 in addition to watching this lecture.

2 *Introduction*

Mixture models integrate multiple data generating processes into a single model. This is especially useful in cases where the data alone don't allow us to fully identify which observations belong to which process. Mixture models are important in cognitive science because many theories of cognition assume that the behavior of subjects in certain tasks is determined by an interplay of different cognitive processes. It is important to stress that a mixture distribution of observations is an *assumption* of the latent process developing trial by trial based on a given theory—it doesn't necessarily represent the true generative process. The role of Bayesian modeling is to help us understand the extent to which this assumption is well-founded, by using posterior predictive checks and by comparing different models.

We focus here on the case where we have only two components; each component represents a distinct cognitive process based on the domain knowledge of the researcher. The vector $\mathbf{z}$ serves as a latent *indicator variable* that indicates which of the mixture components an observation y_n belongs to ($n = 1, \ldots, N$ is the number of data points). We assume two components, and thus each z_n can be either 0 or 1 (this will allows us to generate z_n with a Bernoulli distribution). We also assume two different generative processes, p_1 and p_2, which generate

different distributions of the observations based on a vector of parameters indicated by Θ_1 and Θ_2, respectively. These two processes occur with probability θ and $1 - \theta$, and each observation is generated as follows:

$$
\begin{aligned}
z_n &\sim Bernoulli(\theta) \\
y_n &\sim \begin{cases} p_1(\Theta_1), & \text{if } z_n = 1 \, (\#eq : mixz) \\ p_2(\Theta_2), & \text{if } z_n = 0 \end{cases}
\end{aligned} \tag{1}
$$

We focus on only two components because this type of model is already hard to fit and, as we show in this chapter, it requires plenty of prior information to be able to sample from the posterior in most applied situations. However, the approach presented here can in principle be extended to a larger number of mixtures by replacing the Bernoulli distribution with a categorical one. This can be done if the number of components in the mixture is finite; the number of components is determined by the researcher.

In order to fit this model, we need to estimate the posterior of each of the parameters contained in the vectors Θ_1 and Θ_2 (intercepts, slopes, group-level effects, etc.), the probability θ, and the indicator variable that corresponds to each observation z_n. One issue that presents itself here is that z_n must be a discrete parameter, and Stan only allows continuous parameters. This is because Stan's algorithm requires the derivatives of the (log) posterior distribution with respect to all parameters, and discrete parameters are not differentiable (since they have "breaks"). In probabilistic programming languages like WinBUGS, JAGS, PyMC3, and Turing, discrete parameters are possible to use; but not in Stan. In Stan, we can circumvent this issue by marginalizing out the indicator variable z.[1] If p_1 appears in the mixture with probability θ, and p_2 with probability $1 - \theta$, then the joint likelihood is defined as a function of Θ (which concatenates the mixing probability, θ, and the parameters of the p_1 and p_2, Θ_1 and Θ_2), and importantly z_n "disappears":

[1] See chapter 1 for a review on the concept of marginalization.

$$p(y_n|\Theta) = \theta \cdot p_1(y_n|\Theta_1) + (1-\theta) \cdot p_2(y_n|\Theta_2) \qquad (2)$$

The intuition behind this formula is that each likelihood function, p_1, p_2 is weighted by its probability of being the relevant generative process. For our purposes, it suffices to say that marginalization works; the reader interested in the mathematics behind marginalization is directed to the further reading section at the end of the chapter.[2]

Even though Stan cannot fit a model with the discrete indicator of the latent class **z** that we used in Equation @ref(eq:mixz), this equation will prove very useful when we want to generate synthetic data.

In the following sections, we model a well-known phenomenon (i.e., the speed-accuracy trade-off) assuming an underlying finite mixture process. We start from the verbal description of the model, and then implement the model step by step in Stan.

[2] As mentioned above, other probabilistic languages that do not rely on Hamiltonian dynamics (exclusively) are able to deal with this. However, even when sampling discrete parameters is possible, marginalization is more efficient: when z_n is omitted we are fitting a model with n fewer parameters.

3 *A mixture model of the speed-accuracy trade-off: The fast-guess model account*

When we are faced with multiple choices that require an immediate decision, we can speed up the decision at the expense of accuracy and become more accurate at the expense of speed; this is called the speed-accuracy trade-off. The most popular class of models that can incorporate both response times and accuracy, and give an account for the speed-accuracy trade-off is the class of sequential sampling models, which include the drift diffusion model, the linear ballistic accumulator, and the log-normal race model, which we discuss in chapter 18.

However, an alternative model that has been proposed in the past is Ollman's simple fast-guess model. Although it has mostly fallen out of favor, it presents a very simple framework using finite mixture modeling that can also account for the speed-accuracy trade-off. In the next sections, we'll use this model to exemplify the use of finite

mixtures to represent different cognitive processes.

3.1 The global motion detection task

One way to examine the behavior of human and primate subjects when faced with two-alternative forced choices is the detection of the global motion of a random dot kinematogram. In this task, a subject sees a number of random dots on the screen. A proportion of dots move in a single direction (e.g., right) and the rest move in random directions. The subject's goal is to estimate the overall direction of the movement. One of the reasons for the popularity of this task is that it permits the fine-tuning of the difficulty of trials: The task is harder when the proportion of dots that move coherently (the level of *coherence*) is lower; see the Figure below.

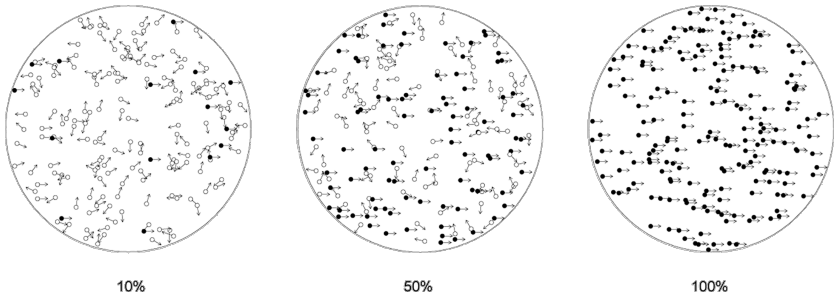

Figure 1: Three levels of difficulty of the global motion detection task. The figures show a consistent movement to the right with three levels of coherence (10%, 50%, and 100%). The subjects see the dots moving in the direction indicated by the arrows. The subjects do not see the arrows and all the dots look identical in the actual task.

The fast-guess model assumes that the behavior in this task (and in any other choice task) is governed by two distinct cognitive processes: (i) a guessing mode, and (ii) a task-engaged mode. In the guessing mode, responses are fast and accuracy is at chance level. In the task-engaged mode, responses are slower and accuracy approaches 100%. This means that intermediate values of response times and accuracy can only be achieved by mixing responses from the two modes. Further assumptions of this model are that response times depend on the difficulty of the choice, and that the probability of being on one of the two states depend on the speed incentives during the instructions.

To simplify matters, we ignore the possibility that the accuracy of the choice is also affected by the difficulty of

the choice. Also, we ignore the possibility that subjects might be biased to one specific response in the guessing mode, but there is an exercise on this.

3.2 *The data set*

We implement the assumptions behind Ollman's fast-guess model and examine its fit to data of a global motion detection task.

The data set contains approximately 2800 trials of each of the 20 subjects participating in a global motion detection task and can be found in `df_dots` in the `bcogsci` package. There were two level of coherence, yielding hard and easy trials (`diff`), and the trials where done under instructions that emphasized either accuracy or speed (`emphasis`). More information about the data set can be found by accessing the documentation for the data set (by typing `?df_dots` on the R command line, assuming that the `bcogsci` package is installed).

```
data("df_dots")
df_dots
```

```
## # A tibble: 56,097 x 12
##     subj diff  emphasis    rt   acc fix_dur stim  resp  trial block block_trial
##    <int> <chr> <chr>    <dbl> <int>   <dbl> <chr> <chr> <int> <int>       <int>
##  1     1 easy  speed      482     1   0.738 R     R         1     6           1
##  2     1 hard  speed      602     1   0.784 R     R         2     6           2
##  3     1 hard  speed      381     1   0.651 R     R         3     6           3
##  4     1 hard  speed      584     1   0.781 R     R         4     6           4
##  5     1 hard  speed      464     0   0.585 L     R         5     6           5
##  6     1 easy  speed      553     1   0.988 R     R         6     6           6
##  7     1 easy  speed      500     1   0.68  R     R         7     6           7
##  8     1 easy  speed      364     1   0.761 L     L         8     6           8
##  9     1 easy  speed      411     1   0.674 L     L         9     6           9
## 10     1 easy  speed      448     1   0.515 L     L        10     6          10
## # i 56,087 more rows
## # i 1 more variable: bias <chr>
```

We might think that if the fast-guess model were true, we would see a bimodal distribution, when we plot a

histogram of the data. Unfortunately, when two similar distributions are mixed, we won't necessarily see any apparent bimodality; see the Figure below.

```
ggplot(df_dots, aes(rt)) +
  geom_histogram()
```

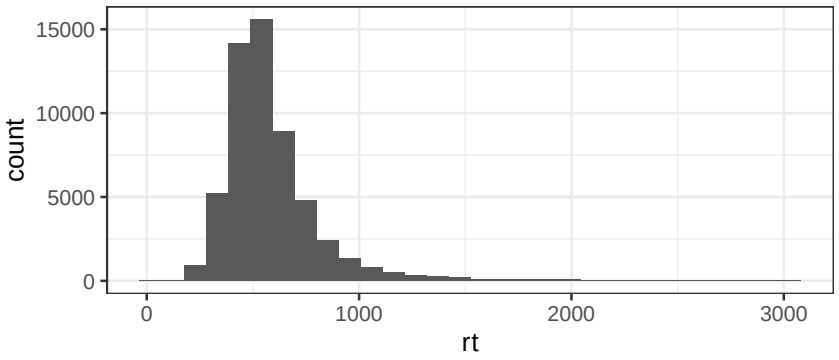

Figure 2: Distribution of response times in the data of the global motion detection task.

However, the Figure below reveals that incorrect responses were generally faster, and this was especially true when the instructions emphasized accuracy.

```
ggplot(df_dots, aes(x = factor(acc), y = rt)) +
  geom_point(position = position_jitter(width = .4, height = 0),
             alpha = 0.5) +
  facet_wrap(diff ~ emphasis) +
  xlab("Accuracy") +
  ylab("Response time")
```

3.3 *A very simple implementation of the fast-guess model*

The description of the model makes it clear that an ideal subject who never guesses has a response time that depends only on the difficulty of the trial. As we did in previous chapters, we assume that response times are log-normally distributed, and for simplicity we start by modeling the behavior of a single subject:

$$rt_n \sim LogNormal(\alpha + \beta \cdot x_n, \sigma) \tag{3}$$

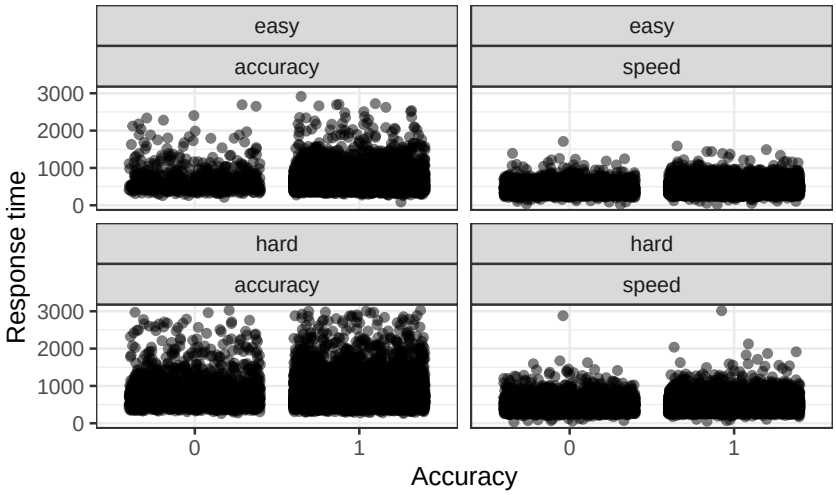

Figure 3: The distribution of response times by accuracy in the data of the global motion detection task.

In the previous equation, x is larger for difficult trials. If we center x, α represents the average logarithmic transformed response time for a subject engaged in the task, and β is the effect of trial difficulty on log-response time. We assume a non-deterministic process, with a noise parameter σ.

Alternatively, a subject that guesses in every trial would show a response time distribution that is independent of the difficulty of the trial:

$$rt_n \sim LogNormal(\gamma, \sigma_2) \tag{4}$$

Here γ represents the the average logarithmic transformed response time when a subject only guesses. We assume that responses from the guessing mode might have a different noise component than from the task-engaged mode.

The fast-guess model makes the assumption that during a task, a single subject would behave in these two ways: They would be engaged in the task a proportion of the trials and would guess on the rest of the trials. This means that for a single subject, there is an underlying probability of being engaged in the task, p_{task}, that determines whether they are actually choosing ($z = 1$) or guessing ($z = 0$):

$$z_n \sim Bernoulli(p_{task}) \tag{5}$$

The value of the parameter z in every trial determines the behavior of the subject. This means that the distribution that we observe is a mixture of the two distributions presented before:

$$rt_n \sim \begin{cases} LogNormal(\alpha + \beta \cdot x_n, \sigma), & \text{if } z_n = 1 \\ LogNormal(\gamma, \sigma_2), & \text{if } z_n = 0 \end{cases} \tag{6}$$

In order to have a Bayesian implementation, we also need to define some priors. We use priors that encode what we know about response time experiments. These priors can be considered regularizing priors. One can verify this by performing prior predictive checks. As we increase the complexity of our models, it's worth spending some time designing more realistic priors. These will speed up computation and in some cases they will be crucial for solving convergence problems.

$$\begin{aligned} \alpha &\sim Normal(6, 1) \\ \beta &\sim Normal(0, 0.1) \\ \sigma &\sim Normal_+(0.5, 0.2) \end{aligned} \tag{7}$$

$$\begin{aligned} \gamma &\sim Normal(6, 1) \\ \sigma_2 &\sim Normal_+(0.5, 0.2) \end{aligned} \tag{8}$$

For now, we allow all values for the probability of having an engaged response equal likelihood; we achieve this by setting the following prior to p_{task}:

$$p_{task} \sim Beta(1, 1) \tag{9}$$

This represents a flat, uninformative prior over the probability parameter p_{task}.

Before we fit our model to the real data, we generate synthetic data to make sure that our model is working as expected.

We first define the number of observations, predictors, and fixed point values for each of the parameters. We

assume 1000 observations and two levels of difficulty, x, coded -0.5 (easy) and 0.5 (hard). The point values chosen for the parameters are relatively realistic (based on our previous experience on response time experiments). Although in the priors we try to encode the range of possible values for the parameters, in this simulation we assume only one instance of this possible range:

```
N <- 1000
# level of difficulty:
x <- c(rep(-0.5, N/2), rep(0.5, N/2))
# Parameters true point values:
alpha <- 5.8
beta <- 0.05
sigma <- 0.4
sigma2 <- 0.5
gamma <- 5.2
p_task <- 0.8
# Median time
c("engaged" = exp(alpha), "guessing" = exp(gamma))
```

```
##  engaged guessing
## 330.2996 181.2722
```

For generating a mixture of response times, we use the indicator of a latent class, z.

```
z <- rbern(n = N, prob = p_task)
rt <- if_else(z == 1,
              rlnorm(N,
                     meanlog = alpha + beta * x,
                     sdlog = sigma),
              rlnorm(N,
                     meanlog = gamma,
                     sdlog = sigma2))
df_dots_simdata1 <- tibble(trial = 1:N, x = x, rt = rt)
```

We verify that our simulated data is realistic, that is, it's in the same range as the original data; see the Figure below.

```
ggplot(df_dots_simdata1, aes(rt)) +
  geom_histogram()
```

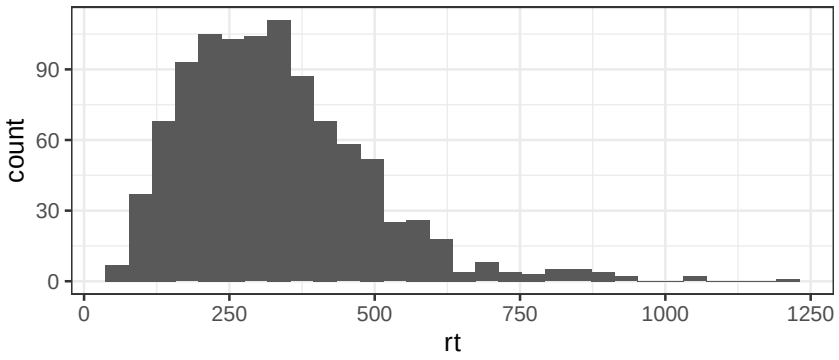

Figure 4: Response times in the simulated data that follows the fast-guess model.

To implement the mixture model in Stan, the discrete parameter z needs to be marginalized out:

$$p(rt_n|\Theta) = p_{task} \cdot LogNormal(rt_n|\alpha + \beta \cdot x_n, \sigma) + (1 - p_{task}) \cdot LogNormal(rt_n|\gamma, \sigma_2) \tag{10}$$

In addition, Stan requires the likelihood to be defined in log-space:

$$\log(p(rt|\Theta)) = \log(p_{task} \cdot LogNormal(rt_n|\alpha + \beta \cdot x_n, \sigma) + (1 - p_{task}) \cdot LogNormal(rt_n|\gamma, \sigma_2)) \tag{11}$$

A "naive" implementation in Stan would look like the following (recall that `_lpdf` functions provide log-transformed densities):

```
target += log(
        p_task * exp(lognormal_lpdf(rt[n] | alpha + x[n] * beta, sigma)) +
              (1-p_task) * exp(lognormal_lpdf(rt[n] | gamma, sigma2)));
```

However, we need to take into account that $log(A \pm B)$ can be numerically unstable [i.e., prone to underflow/overflow]. Stan provides several functions to deal with different special cases of logarithms of sums and differences. Here we need `log_sum_exp(x, y)` that corresponds to `log(exp(x) + exp(y))` and `log1m(x)` that corresponds to `log(1-x)`.

First, we need to take into account that the first summand of the logarithm, `p_task * exp(lognormal_lpdf(rt[n] | alpha + x[n] * beta, sigma))` corresponds to `exp(x)`, and the second one, `(1-p_task) * exp(lognormal_lpdf(rt[n] | gamma, sigma2))` to `exp(y)` in `log_sum_exp(x, y)`. This means that we need to first apply the logarithm to each of them to use them as arguments of `log_sum_exp(x, y)`:

```
target += log_sum_exp(
              log(p_task) + lognormal_lpdf(rt[n] | alpha +
              x[n] * beta, sigma),
              log(1-p_task) + lognormal_lpdf(rt[n] | gamma, sigma2));
```

Now we can just replace `log(1-p_task)` by the more stable `log1m(p_task)`:

```
target += log_sum_exp(
               log(p_task) + lognormal_lpdf(rt[n] | alpha +
               x[n] * beta, sigma),
               log1m(p_task) + lognormal_lpdf(rt[n] | gamma, sigma2));
```

The complete model (`mixture_rt.stan`) is shown below:

```
data {
  int<lower = 1> N;
  vector[N] x;
  vector[N] rt;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
  real gamma; //guessing
  real<lower = 0> sigma2;
  real<lower = 0, upper = 1> p_task;
}
model {
  // priors for the task component
  target += normal_lpdf(alpha | 6, 1);
  target += normal_lpdf(beta | 0, 0.1);
```

```
  target += normal_lpdf(sigma | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  // priors for the guessing component
  target += normal_lpdf(gamma | 6, 1);
  target += normal_lpdf(sigma2 | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += beta_lpdf(p_task | 1, 1);
  // likelihood
  for(n in 1:N)
    target +=
      log_sum_exp(log(p_task) +
                  lognormal_lpdf(rt[n] | alpha + x[n] * beta, sigma),
                  log1m(p_task) +
                  lognormal_lpdf(rt[n] | gamma, sigma2));
}
```

Call the Stan model `mixture_rt.stan`, and fit it to the simulated data. First, we set up the simulated data as a list structure:

```
ls_dots_simdata <- list(N = N, rt = rt, x = x)
```

Then fit the model:

```
mixture_rt <- system.file("stan_models",
                          "mixture_rt.stan",
                          package = "bcogsci")
fit_mix_rt <- stan(mixture_rt, data = ls_dots_simdata)
```

```
## Warning: The largest R-hat is 1.74, indicating chains have not mixed.
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#r-hat
##
##  Warning: Bulk Effective Samples Size (ESS) is too low, indicating
## posterior means and medians may be unreliable. Running the chains for
## more iterations may help. See
## https://mc-stan.org/misc/warnings.html#bulk-ess
##
##  Warning: Tail Effective Samples Size (ESS) is too low, indicating
## posterior variances and tail quantiles may be unreliable. Running the
## chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#tail-ess
```

There are a lot of warnings, the Rhats are too large, and number of effective samples is too low:

```
print(fit_mix_rt)
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##            mean se_mean   sd     2.5%      25%      50%      75%    97.5% n_eff
## alpha      5.55    0.16 0.28     4.99     5.39     5.55     5.69     6.00     3
## beta       0.02    0.03 0.07    -0.12    -0.03     0.01     0.06     0.16     6
## sigma      0.44    0.07 0.11     0.21     0.38     0.49     0.52     0.56     3
## gamma      5.81    0.11 0.17     5.43     5.70     5.86     5.92     6.01     2
## sigma2     0.39    0.05 0.09     0.22     0.32     0.38     0.45     0.54     3
## p_task     0.46    0.04 0.21     0.12     0.27     0.45     0.65     0.83    24
## lp__   -6386.51    0.47 1.94 -6391.16 -6387.58 -6386.25 -6385.17 -6383.24    17
##        Rhat
## alpha  1.74
## beta   1.24
## sigma  2.14
## gamma  2.31
## sigma2 1.76
## p_task 1.08
## lp__   1.09
##
## Samples were drawn using NUTS(diag_e) at Tue Mar 31 15:49:57 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

The traceplots show clearly that the chains aren't mixing; see the Figure below.

```
traceplot(fit_mix_rt) +
  scale_x_continuous(breaks = NULL)
```

The problem with this model is that the mixture components (i.e., the fast-guesses and the engaged mode) are underlyingly exchangeable and thus the posterior is multimodal and the model does not converge. Each

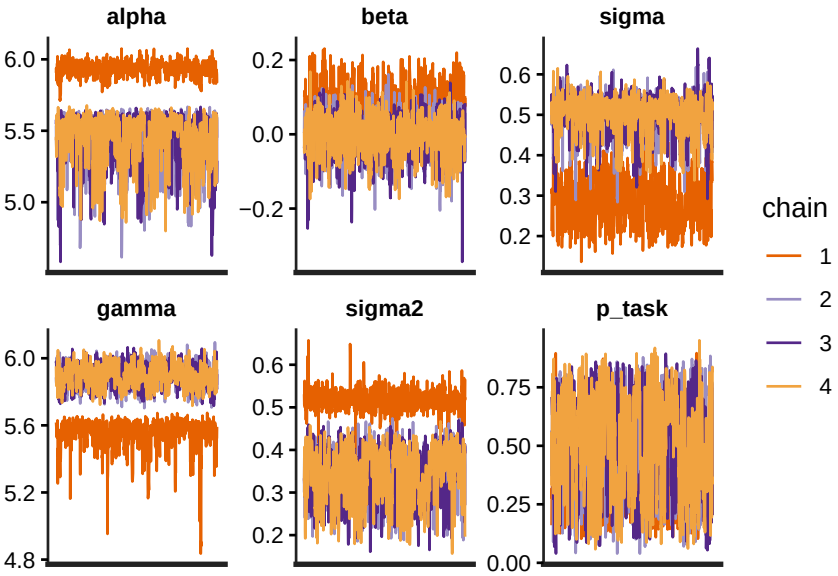

Figure 5: Traceplots from the model fit to simulated data.

chain (and each iteration) doesn't know how each component was identified by the rest of the chains (e.g., in some chains and iterations, $z = 0$ corresponds to fast guesses, whereas in other cases, it corresponds to deliberate responses). However, we do have information that can identify the components: According to the theoretical model, we know that the average response in the engaged mode, represented by α, should be slower than the average response in the guessing mode, γ.

Even though the theoretical model assumes that guesses are faster than engaged responses, this is not explicit in our computational model. That is, our model lacks some of the theoretical information that we have, namely that the distribution of engaged response times should be slower than the distribution of guessing times. This can be encoded with *order constraints* or *inequality constraints*: a strong prior for γ, where we assume that the upper bound of its prior distribution is truncated at the value α:

$$\gamma \sim Normal(6, 1), \text{for } \gamma < \alpha \tag{12}$$

This would be enough to make the model converge. Another softer constraint that we could add to our im-

plementation is the assumption that subjects are generally more likely to be trying to do the task than just guessing. If this assumption is correct, we also improve the accuracy of our estimation of the posterior of the model. (The opposite is also true: If subjects are not trying to do the task, this assumption will be unwarranted and our prior information will lead us further from the "true" values of the parameters). The following prior has the probability density concentrated near 1.

$$p_{task} \sim Beta(8, 2) \tag{13}$$

Plotting this prior confirms where most of the probability mass lies; see the Figure below.

```
plot(function(x) dbeta(x, 8, 2), ylab = "density" , xlab= "probability")
```

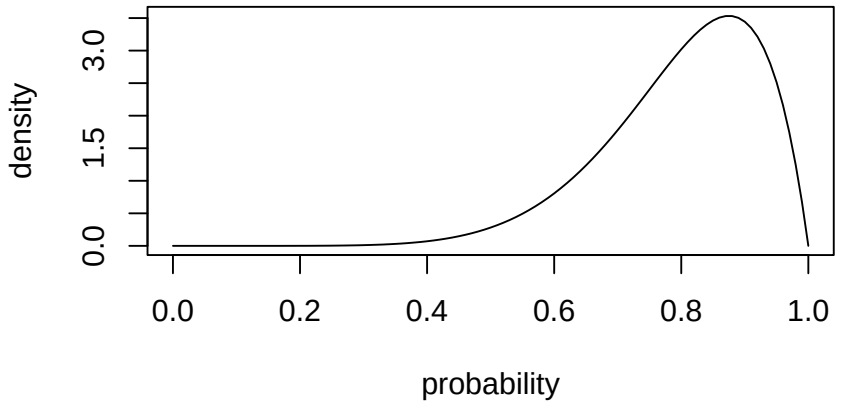

Figure 6: A density plot for the Beta(8,2) prior on p_{task}.

The Stan code for this model is shown below as `mixture_rt2.stan`.

```
data {
  int<lower = 1> N;
  vector[N] x;
  vector[N] rt;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
  real<upper = alpha> gamma;
```

```
  real<lower = 0> sigma2;
  real<lower = 0, upper = 1> p_task;
}
model {
  target += normal_lpdf(alpha | 6, 1);
  target += normal_lpdf(beta | 0, 0.3);
  target += normal_lpdf(sigma | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(gamma | 6, 1) -
    normal_lcdf(alpha | 6, 1);
  target += normal_lpdf(sigma2 | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += beta_lpdf(p_task | 8, 2);
  for(n in 1:N)
    target +=
      log_sum_exp(log(p_task) +
                  lognormal_lpdf(rt[n] | alpha + x[n] * beta, sigma),
                  log1m(p_task) +
                  lognormal_lpdf(rt[n] | gamma, sigma2)) ;
}
```

Once we change the upper bound of `gamma` in the `parameters` block, we also need to truncate the distribution in the `model` block by correcting the PDF with its CDF. This correction is carried out using the CDF because we are truncating the distribution at the right-hand side; recall that earlier we used the complement of the CDF when we truncate a distribution at the left-hand side); see the online section on truncation.

```
  target += normal_lpdf(gamma | 6, 1) -
    normal_lcdf(alpha | 6, 1);
```

Fit this model (call it `mixture_rt2.stan`) to the same simulated data set that we used before:

```
mixture_rt2 <- system.file("stan_models",
                           "mixture_rt2.stan",
                           package = "bcogsci")
fit_mix_rt2 <- stan(mixture_rt2, data = ls_dots_simdata)
```

Now the summaries and traceplots look fine; see the Figure below.

```
print(fit_mix_rt2)
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##            mean se_mean   sd     2.5%      25%      50%      75%    97.5% n_eff
## alpha      5.84    0.00 0.05     5.75     5.81     5.84     5.88     5.95  1384
## beta       0.05    0.00 0.04    -0.03     0.02     0.05     0.07     0.14  1468
## sigma      0.39    0.00 0.03     0.30     0.37     0.39     0.41     0.44  1013
## gamma      5.27    0.01 0.17     4.93     5.15     5.28     5.40     5.57  1126
## sigma2     0.47    0.00 0.06     0.36     0.43     0.47     0.52     0.58  1174
## p_task     0.69    0.00 0.14     0.35     0.61     0.71     0.80     0.91   983
## lp__   -6387.24    0.06 1.93 -6391.81 -6388.32 -6386.89 -6385.78 -6384.56  1091
##        Rhat
## alpha     1
## beta      1
## sigma     1
## gamma     1
## sigma2    1
## p_task    1
## lp__      1
##
## Samples were drawn using NUTS(diag_e) at Tue Mar 31 15:50:28 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
traceplot(fit_mix_rt2) +
  scale_x_continuous(breaks = NULL)
```

3.4 *A multivariate implementation of the fast-guess model*

A problem with the previous implementation of the fast-guess model is that we ignore the accuracy information in the data. We can implement a version that is closer to the verbal description of the model: In particular, we also

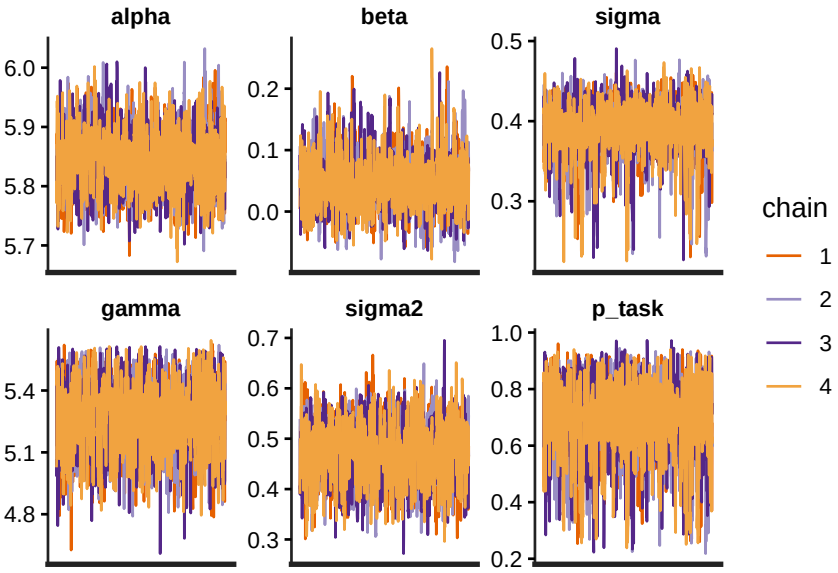

Figure 7: Traceplots from the model fit to simulated data.

want to model the fact that accuracy is at chance level in the fast-guessing mode and that accuracy approaches 100% during the task-engaged mode.

This means that the mixture affects two pairs of distributions:

$$z_n \sim Bernoulli(p_{task}) \tag{14}$$

The response time distribution

$$rt_n \sim \begin{cases} LogNormal(\alpha + \beta \cdot x_n, \sigma), & \text{if } z_n = 1 \\ LogNormal(\gamma, \sigma_2), & \text{if } z_n = 0 \end{cases} (\#eq : dismix2) \tag{15}$$

and an accuracy distribution

$$acc_n \sim \begin{cases} Bernoulli(p_{correct}), & \text{if } z_n = 1 \\ Bernoulli(0.5), & \text{if } z_n = 0 \end{cases} (\#eq : dismix3) \tag{16}$$

We have a new parameter $p_{correct}$, which represent the probability of making a correct answer in the engaged mode. The verbal description says that it is closer to 100%, and here we have the freedom to choose whatever

prior we believe represents for us values that are close to 100% accuracy. We translate this belief into a prior as follows; our prior choice is relatively informative but does not impose a hard constraint; if a subject consistently shows relatively low (or high) accuracy, $p_{correct}$ will change accordingly:

$$p_{correct} \sim Beta(995, 5) \tag{17}$$

In our simulated data, we assume that the global motion detection task is done by a very accurate subject, with an accuracy of 99.9%.

First, simulate response times, as done earlier:

```
N <- 1000
x <- c(rep(-0.5, N/2), rep(0.5, N/2)) # difficulty
alpha <- 5.8
beta <- 0.05
sigma <- 0.4
sigma2 <- 0.5
gamma <- 5.2 # fast guess location
p_task <- 0.8 # prob of being on task
z <- rbern(n = N, prob = p_task)
rt <- if_else(z == 1,
              rlnorm(N,
                     meanlog = alpha + beta * x,
                     sdlog = sigma),
              rlnorm(N,
                     meanlog = gamma,
                     sdlog = sigma2))
```

Simulate accuracy and include both response times and accuracy in the simulated data set:

```
p_correct <- 0.999
acc <- ifelse(z, rbern(n = N, p_correct),
                  rbern(n = N, 0.5))
df_dots_simdata3 <- tibble(trial = 1:N,
                           x = x,
                           rt = rt,
```

```
                    acc = acc) %>%
  mutate(diff = if_else(x == 0.5, "hard", "easy"))
```

Plot the simulated data in the Figure shown. This time we can see the effect of task difficulty on the simulated response times and accuracy:

```
ggplot(df_dots_simdata3, aes(x = factor(acc), y = rt)) +
  geom_point(position = position_jitter(width = 0.4, height = 0),
             alpha = 0.5) +
  facet_wrap(diff ~ .) +
  xlab("Accuracy") +
  ylab("Response time")
```

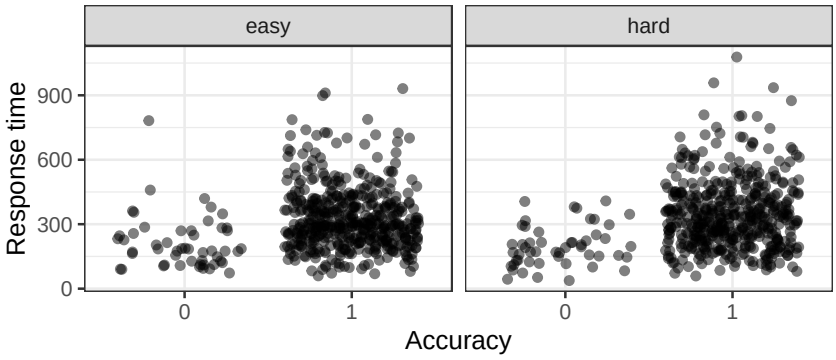

Figure 8: Response times by accuracy, accounting for task difficulty in the simulated data that follows the fast-guess model.

Next, we need to marginalize out the discrete parameters from the joint distribution of accuracy and response times. However, we assume that conditional on the latent indicator parameter z, response times and accuracy are independent. For this reason, we can multiply the likelihoods for `rt` and `acc` within each component.

$$
\begin{aligned}
p(rt, acc|\Theta) = & p_{task} \cdot \\
& LogNormal(rt_n|\alpha + \beta \cdot x_n, \sigma) \cdot \\
& Bernoulli(acc_n|p_{correct}) \\
& + \\
& (1 - p_{task}) \cdot \\
& LogNormal(rt_n|\gamma, \sigma_2) \cdot \\
& Bernoulli(acc_n|0.5)
\end{aligned}
\tag{18}
$$

In log-space:

$$
\begin{aligned}
\log(p(rt, acc|\Theta)) = \log(\exp(& \\
& \log(p_{task}) + \\
& \log(LogNormal(rt_n|\alpha + \beta \cdot x_n, \sigma)) + \\
& \log(Bernoulli(acc_n|p_{correct}))) \\
& + \\
\exp(& \\
& \log(1 - p_{task}) + \\
& \log(LogNormal(rt_n|\gamma, \sigma_2)) + \\
& \log(Bernoulli(acc_n|0.5))) \\
&)
\end{aligned}
\tag{19}
$$

Our model translates to the following Stan code (`mixture_rtacc.stan`):

```
data {
  int<lower = 1> N;
  vector[N] x;
  vector[N] rt;
  array[N] int acc;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
  real<upper = alpha> gamma;
  real<lower = 0> sigma2;
  real<lower = 0, upper = 1> p_correct;
  real<lower = 0, upper = 1> p_task;
}
model {
  target += normal_lpdf(alpha | 6, 1);
  target += normal_lpdf(beta | 0, 0.3);
  target += normal_lpdf(sigma | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(gamma | 6, 1) -
    normal_lcdf(alpha | 6, 1);
```

```
  target += normal_lpdf(sigma2 | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += beta_lpdf(p_correct | 995, 5);
  target += beta_lpdf(p_task | 8, 2);
  for(n in 1:N){
    target +=
      log_sum_exp(log(p_task) +
                  lognormal_lpdf(rt[n] | alpha + x[n] * beta, sigma) +
                  bernoulli_lpmf(acc[n] | p_correct),
                  log1m(p_task) +
                  lognormal_lpdf(rt[n] | gamma, sigma2) +
                  bernoulli_lpmf(acc[n] | 0.5));
  }
}
```

Next, set up the data in list format:

```
ls_dots_simdata <- list(N = N,
                        rt = rt,
                        x = x,
                        acc = acc)
```

Then fit the model:

```
mixture_rtacc <- system.file("stan_models",
                             "mixture_rtacc.stan",
                             package = "bcogsci")
fit_mix_rtacc <- stan(mixture_rtacc, data = ls_dots_simdata)
```

We see that our model can be fit to both response times and accuracy, and its parameters estimates have sensible values (given the fixed parameters we used to generate our simulated data).

```
print(fit_mix_rtacc)
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##               mean se_mean   sd     2.5%      25%      50%      75%    97.5%
## alpha         5.79    0.00 0.02     5.76     5.78     5.79     5.80     5.83
```

```
## beta          0.05    0.00 0.03    -0.02     0.03     0.05     0.07     0.11
## sigma         0.40    0.00 0.01     0.37     0.39     0.40     0.41     0.42
## gamma         5.15    0.00 0.05     5.05     5.12     5.15     5.19     5.25
## sigma2        0.51    0.00 0.03     0.45     0.49     0.51     0.53     0.58
## p_correct     1.00    0.00 0.00     0.99     0.99     1.00     1.00     1.00
## p_task        0.80    0.00 0.02     0.76     0.79     0.80     0.81     0.83
## lp__      -6639.47    0.05 1.95 -6644.17 -6640.56 -6639.09 -6638.02 -6636.71
##           n_eff Rhat
## alpha      3935    1
## beta       5573    1
## sigma      5074    1
## gamma      4107    1
## sigma2     4843    1
## p_correct  4587    1
## p_task     4981    1
## lp__       1783    1
##
## Samples were drawn using NUTS(diag_e) at Tue Mar 31 15:50:54 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

We will evaluate the recovery of the parameters more carefully when we deal with the hierarchical version of the fast-guess model. Before we extend this model hierarchically, let us also take into account the instructions given to the subjects.

3.5 *An implementation of the fast-guess model that takes instructions into account*

The actual global motion detection experiment that we started with has another manipulation that can help us to evaluate better the fast-guess model. In some trials, the instructions emphasized accuracy (e.g., "Be as accurate as possible.") and in others speed (e.g., "Be as fast as possible."). The fast-guess model also assumes that the probability of being in one of the two states depends on the speed incentives given during the instructions. This entails that now p_{task} depends on the instructions x_2, where

we encode a speed incentive with -0.5 and an accuracy incentive with 0.5. Essentially, we need to fit the following regression:

$$\alpha_{task} + x_2 \cdot \beta_{task} \tag{20}$$

As we did with MPT models in the MPT chapter, we need to bound the previous regression between 0 and 1; we achieve this using the logistic or inverse logit function:

$$p_{task} = logit^{-1}(\alpha_{task} + x_2 \cdot \beta_{task}) \tag{21}$$

This means that we need to interpret $\alpha_{task} + x_2 \cdot \beta_{task}$ in log-odds space, which has the range $(-\infty, \infty)$ rather than the limited range of probability space, $[0, 1]$.

The likelihood (defined before) now depends on the value of x_2 for the specific row:

$$z_n \sim Bernoulli(p_{task_n}) \tag{22}$$

A response time distribution is defined:

$$rt_n \sim \begin{cases} LogNormal(\alpha + \beta \cdot x_n, \sigma), & \text{if } z_n = 1 \\ LogNormal(\gamma, \sigma_2), & \text{if } z_n = 0 \end{cases} \tag{23}$$

and an accuracy distribution is defined as well:

$$acc_n \sim \begin{cases} Bernoulli(p_{correct}), & \text{if } z_n = 1 \\ Bernoulli(0.5), & \text{if } z_n = 0 \end{cases} \tag{24}$$

The only further change in our model is that rather than a prior on p_{task}, we now need priors for α_{task} and β_{task}, which are on the log-odds scale.

For β_{task}, we assume an effect that can be rather large and we won't assume a direction a priori (for now):

$$\beta_{task} \sim Normal(0, 1) \tag{25}$$

This means that the subject could be affected by the instructions in the expected way, with an increased probability to be task-engaged, leading to better accuracy when

the instructions emphasize accuracy ($\beta_{task} > 0$). Alternatively, the subject might behave in an unexpected way, with a decreased probability to be task-engaged, leading to worse accuracy when the instructions emphasize accuracy ($\beta_{task} < 0$). The latter situation, $\beta_{task} < 0$, could represent the instructions being misunderstood. It's certainly possible to include priors that encode the expected direction of the effect instead: $Normal_+(0, 1)$. Unless there is a compelling reason to constrain the prior in this way, following Cromwell's rule in the online chapter on priors, we leave open the possibility of the β parameter having negative values.

How can we choose a prior for α_{task} that encodes the same information that we had in the previous model in p_{task}? One possibility is to create an auxiliary parameter p_{btask}, that represents the baseline probability of being engaged in the task, with the same prior that we use in the previous section, and then transform it to an unconstrained space for our regression with the logit function:

$$\begin{aligned} p_{btask} &\sim Beta(8, 2) \\ \alpha_{task} &= logit(p_{btask}) \end{aligned} \tag{26}$$

To verify that our priors make sense, in the Figure below we plot the difference in prior predicted probability of being engaged in the task under the two emphasis conditions:

```
Ns <- 1000 # number of samples for the plot
# Priors
p_btask <- rbeta(n = Ns, shape1 = 8, shape2 = 2)
beta_task <- rnorm(n = Ns, mean = 0, sd = 1)
# Predicted probability of being engaged
p_task_easy <- plogis(qlogis(p_btask) + 0.5 * beta_task)
p_task_hard <- plogis(qlogis(p_btask) + -0.5 * beta_task)
# Predicted difference
diff_p_pred <- tibble(diff = p_task_easy - p_task_hard)
```

```
diff_p_pred %>%
  ggplot(aes(diff)) +
  geom_histogram()
```

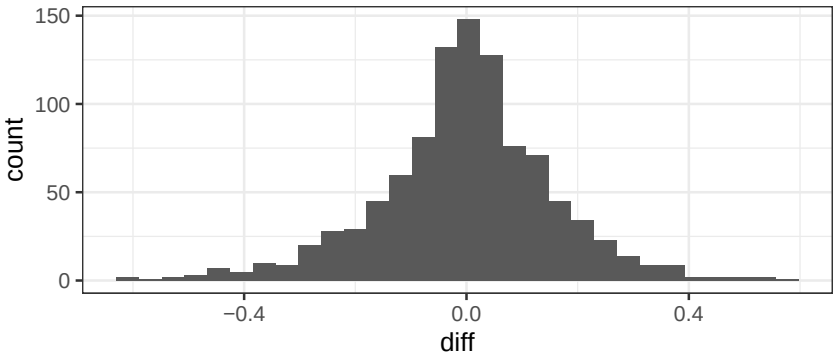

Figure 9: The difference in prior predicted probability of being engaged in the task under the two emphasis conditions for the simulated data that follows the fast-guess model.

The figure shows that we are predicting a priori that the difference in p_{task} will tend to be smaller than ± 0.3, which seems to make sense intuitively. If we had more information about the likely range of variation, we could of course have adapted the prior to reflect that belief.

We are ready to generate a new data set, by deciding on some fixed values for β_{task} and p_{btask}:

```
N <- 1000
x <- c(rep(-0.5, N/2), rep(0.5, N/2)) # difficulty
x2 <- rep(c(-0.5, 0.5), N/2) # instructions
# Verify that the predictors are crossed in a 2x2 design :
predictors <- tibble(x, x2)
xtabs(~ x + x2, predictors)
```

```
##       x2
## x      -0.5 0.5
##   -0.5  250 250
##   0.5   250 250
```

```
alpha <- 5.8
beta <- 0.05
sigma <- 0.4
sigma2 <- 0.5
gamma <- 5.2
p_correct <- 0.999
# New true point values:
p_btask <- 0.85
beta_task <- 0.5
# Generate data:
```

```
alpha_task <- qlogis(p_btask)
p_task <- plogis(alpha_task + x2 * beta_task)
z <- rbern(N, prob = p_task)
rt <- ifelse(z,
             rlnorm(N, meanlog = alpha + beta * x, sdlog = sigma),
             rlnorm(N, meanlog = gamma, sdlog = sigma2))
acc <- ifelse(z, rbern(N, p_correct),
                 rbern(N, 0.5))
df_dots_simdata4 <- tibble(trial = 1:N,
                           x = x,
                           rt = rt,
                           acc = acc,
                           x2 = x2) %>%
          mutate(diff = if_else(x == 0.5, "hard", "easy"),
                 emphasis = ifelse(x2 == 0.5, "accuracy", "speed"))
```

We can generate a plot now where both the difficulty of the task and the instructions are manipulated; see the figure below.

```
ggplot(df_dots_simdata4, aes(x = factor(acc), y = rt)) +
  geom_point(position = position_jitter(width = .4, height = 0),
             alpha = 0.5) +
  facet_wrap(diff ~ emphasis) +
  xlab("Accuracy") +
  ylab("Response time")
```

In the Stan implementation, `log_inv_logit(x)` is applying the logistic (or inverse logit) function to `x` to transform it into a probability and then applying the logarithm; `log1m_inv_logit(x)` is applying the logistic function to `x`, and then applying the logarithm to its complement $(1 - p)$. We do this because rather than having `p_task` in probability space, we have `lodds_task` in log-odds space:

```
real lodds_task = logit(p_btask) + x2[n] * beta_task;
```

The parameter `lodds_task` estimates the mixing probabilities in log-odds:

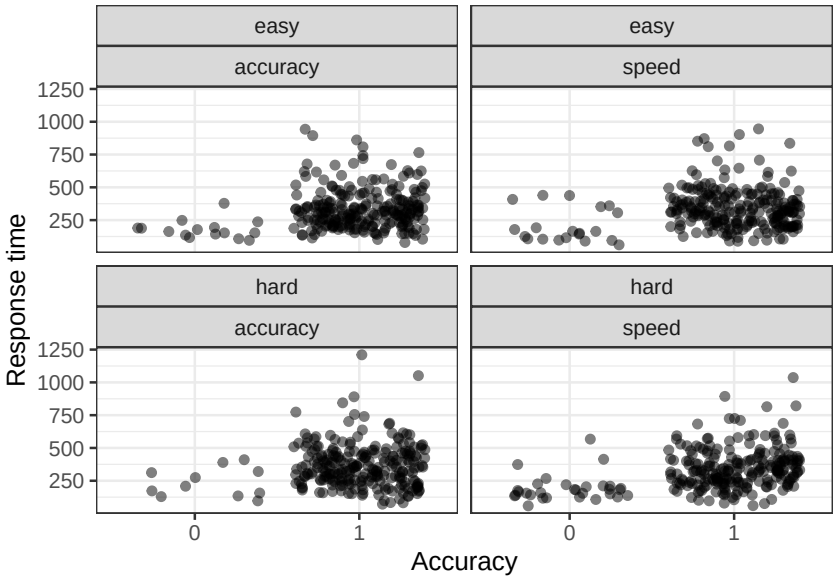

Figure 10: Response times and accuracy by the difficulty of the task and the instructions type for the simulated data that follows the fast-guess model.

```
target += log_sum_exp(log_inv_logit(lodds_task) + ...,
                      log1m_inv_logit(lodds_task) + ...)
```

We also add a `generated quantities` block that can be used for further (prior or posterior) predictive checks. In this block, we do use `z` as an indicator of the latent class (task-engaged mode or fast-guessing mode), since we do not estimate `z`, but rather generate it based on the parameter's posteriors.

We use the dummy variable `onlyprior` to indicate whether we use the data or we only sample from the priors. One can always do the predictive checks in R, transforming the code that we wrote for the simulation into a function, and writing the priors in R. However, it can be simpler to take advantage of Stan output format and rewrite the code in Stan. One downside of this is that the `stanfit` object that stores the model output can become too large for the memory of the computer. Another downside is reduced robustness, as it is more likely that we overlook an error if we only work in Stan rather than re-implementing the code in different programming languages (e.g., R and Stan).

The code shown below is available in the `bcogsci` pack-

age and is called `mixture_rtacc2.stan`.

```
data {
  int<lower = 1> N;
  vector[N] x;
  vector[N] rt;
  array[N] int acc;
  vector[N] x2; //speed or accuracy emphasis
  int<lower = 0, upper = 1> onlyprior;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
  real<upper = alpha> gamma;
  real<lower = 0> sigma2;
  real<lower = 0, upper = 1> p_correct;
  real<lower = 0, upper = 1> p_btask;
  real beta_task;
}
model {
  target += normal_lpdf(alpha | 6, 1);
  target += normal_lpdf(beta | 0, 0.1);
  target += normal_lpdf(sigma | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(gamma | 6, 1)-
  normal_lcdf(alpha | 6, 1);
  target += normal_lpdf(sigma2 | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(beta_task | 0, 1);
  target += beta_lpdf(p_correct | 995, 5);
  target += beta_lpdf(p_btask | 8, 2);
  if(onlyprior != 1)
    for(n in 1:N){
      real lodds_task = logit(p_btask) + x2[n] * beta_task;
      target +=
        log_sum_exp(log_inv_logit(lodds_task)+
                 lognormal_lpdf(rt[n] | alpha + x[n] * beta, sigma) +
                 bernoulli_lpmf(acc[n] | p_correct),
```

```
                log1m_inv_logit(lodds_task) +
                lognormal_lpdf(rt[n] | gamma, sigma2) +
                bernoulli_lpmf(acc[n] | 0.5));
    }
}
generated quantities {
  array[N] real rt_pred;
  array[N] real acc_pred;
  array[N] int z;
  for(n in 1:N){
    real lodds_task = logit(p_btask) + x2[n] * beta_task;
    z[n] = bernoulli_rng(inv_logit(lodds_task));
    if(z[n] == 1){
      rt_pred[n] = lognormal_rng(alpha + x[n] * beta, sigma);
      acc_pred[n] = bernoulli_rng(p_correct);
    } else{
      rt_pred[n] = lognormal_rng(gamma, sigma2);
      acc_pred[n] = bernoulli_rng(0.5);
    }
  }
}
```

Before fitting the model to the simulated data, we perform prior predictive checks.

3.6 Prior predictive checks

Generate prior predictive distributions, by setting `onlyprior` to 1.

```
ls_dots_simdata <- list(N = N,
                        rt = rt,
                        x = x,
                        x2 = x2,
                        acc = acc,
                        onlyprior = 1)
mixture_rtacc2 <- system.file("stan_models",
                              "mixture_rtacc2.stan",
                              package = "bcogsci")
```

```
fit_mix_rtacc2_priors <- stan(mixture_rtacc2,
                              data = ls_dots_simdata)
```

We plot prior predictive distributions of response times as follows, by setting `y = rt` using `ppd_dens_overlay()`.

```
rt_pred <- rstan::extract(fit_mix_rtacc2_priors)$rt_pred
ppd_dens_overlay(rt_pred[1:100,]) +
  coord_cartesian(xlim = c(0, 2000))
```

Some of the predictive data sets contain responses that are too large, and some of them have too much probability mass close to zero, but there is nothing clearly wrong in the prior predictive distributions (considering that the model hasn't "seen" the data yet).

If we want to plot the prior predicted distribution of differences in response time conditioning on task difficulty, we need to define a new function. Then we use the `bayesplot` function `ppc_stat()` that takes as an argument of `stat` any summary function; see sub-figure (a).

```
meanrt_diff <- function(rt){
  mean(rt[x == 0.5]) -
    mean(rt[x == -0.5])
}
ppd_stat(rt_pred, stat = meanrt_diff)
```

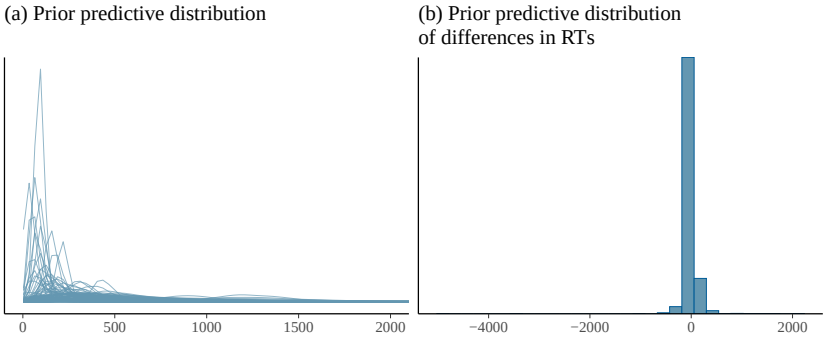

Figure 11: (a) Prior predictive distributions of response times from the fast-guess model. (b) Prior predictive distribution of response time differences, using the same model and prior settings, conditioned on task difficulty.

We find that the range of response times look reasonable. There are, however, always more checks that can be done; examples are plotting other summary statistics, or predictions conditioned on other aspects of the data.

3.7 Fit to simulated data

Fit the model to data, by setting `onlyprior = 0`:

```
ls_dots_simdata <- list(N = N,
                        rt = rt,
                        x = x,
                        x2 = x2,
                        acc = acc,
                        onlyprior = 0)

fit_mix_rtacc2 <- stan(mixture_rtacc2, data = ls_dots_simdata)

print(fit_mix_rtacc2,
      pars = c("alpha", "beta", "sigma", "gamma", "sigma2",
               "p_correct", "p_btask", "beta_task"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##           mean se_mean   sd 2.5%  25%  50%  75% 97.5% n_eff Rhat
## alpha     5.82       0 0.02 5.78 5.80 5.82 5.83  5.85  4712    1
## beta      0.05       0 0.03 0.00 0.03 0.05 0.07  0.11  6991    1
## sigma     0.40       0 0.01 0.38 0.39 0.40 0.41  0.42  5375    1
## gamma     5.12       0 0.06 5.00 5.08 5.12 5.16  5.23  3030    1
## sigma2    0.46       0 0.04 0.37 0.43 0.46 0.48  0.54  3265    1
## p_correct 0.99       0 0.00 0.99 0.99 0.99 1.00  1.00  3390    1
## p_btask   0.86       0 0.02 0.83 0.85 0.86 0.87  0.89  3965    1
## beta_task 0.62       0 0.24 0.16 0.45 0.61 0.78  1.11  5540    1
##
## Samples were drawn using NUTS(diag_e) at Tue Mar 31 16:05:40 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

We see that we fit the model without problems. Before we evaluate the recovery of the parameters more carefully, we implement a hierarchical version of the fast-guess model.

4 *A hierarchical implementation of the fast-guess model*

So far we have evaluated the behavior of one simulated subject. We discussed before (in the context of distributional regression models, in section @ref(sec-distrmodel), and in the MPT modeling chapter that, in principle, every parameter in a model can be made hierarchical. However, this doesn't guarantee that we'll learn anything from the data for those parameters, or that our model will converge. A safe approach here is to start simple, using simulated data. If a model converges on simulated data, it does not guarantee convergence on real data. However, if a model fails to converge on simulated data, it is very likely to fail with real data as well.

For our hierarchical version, we assume that both response times and the effect of task difficulty vary by subject, and that different subjects have different guessing times. This entails the following change to the response time distribution:

$$rt_n \sim \begin{cases} LogNormal(\alpha + u_{subj[n],1} + x_n \cdot (\beta + u_{subj[n],2}), \sigma), & \text{if } z_n = 1 \\ LogNormal(\gamma + u_{subj[n],3}, \sigma_2), & \text{if } z_n = 0 \end{cases} \tag{27}$$

We assume that the three vectors of u (adjustment to the intercept and slope of the task-engaged distribution, and the adjustment to the guessing time distribution) follow a multivariate normal distribution centered on zero. For simplicity and lack of any prior knowledge about this experiment design and method, we assume the same (weakly informative) prior distribution for the three variance components and the same regularizing LKJ prior for the correlation matrix $\mathbf{R_u}$ that contains the three correlations between the adjustments ($\rho_{u_{1,2}}, \rho_{u_{1,3}}, \rho_{u_{2,3}}$):

$$\begin{aligned} \boldsymbol{u} &\sim \mathcal{N}(0, \Sigma_u) \\ \tau_{u_1}, \tau_{u_2}, \tau_{u_3} &\sim Normal_+(0, 0.5) \\ \mathbf{R_u} &\sim LKJcorr(2) \end{aligned} \tag{28}$$

Before we fit the model to the real data set, we simu-

late data again; this time we simulate 20 subjects, each of whom delivers a total of 100 trials (each subject sees 25 trials for each of the four conditions).

```
# Build the simulate data set
N_subj <- 20
N_trials <- 100
# Parameters' true point values
alpha <- 5.8
beta <- 0.05
sigma <- 0.4
sigma2 <- 0.5
gamma <- 5.2
beta_task <- 0.1
p_btask <- 0.85
alpha_task <- qlogis(p_btask)
p_correct <- 0.999
tau_u <- c(0.2, 0.005, 0.3)
rho <- 0.3

## Build the data set here:
N <- N_subj * N_trials
stimuli <- tibble(x = rep(c(rep(-0.5, N_trials / 2),
                            rep(0.5, N_trials / 2)), N_subj),
                  x2 = rep(rep(c(-0.5, 0.5), N_trials / 2), N_subj),
                  subj = rep(1:N_subj, each = N_trials),
                  trial = rep(1:N_trials, N_subj))
stimuli
```

```
## # A tibble: 2,000 x 4
##        x    x2  subj trial
##    <dbl> <dbl> <int> <int>
##  1  -0.5  -0.5     1     1
##  2  -0.5   0.5     1     2
##  3  -0.5  -0.5     1     3
##  4  -0.5   0.5     1     4
##  5  -0.5  -0.5     1     5
##  6  -0.5   0.5     1     6
##  7  -0.5  -0.5     1     7
##  8  -0.5   0.5     1     8
```

```
##  9  -0.5  -0.5     1     9
## 10  -0.5   0.5     1    10
## # i 1,990 more rows
```

```
# Build the correlation matrix for the adjustments u:
Cor_u <- matrix(rep(rho, 9), nrow = 3)
diag(Cor_u) <- 1
Cor_u
```

```
##      [,1] [,2] [,3]
## [1,]  1.0  0.3  0.3
## [2,]  0.3  1.0  0.3
## [3,]  0.3  0.3  1.0
```

```
# Variance covariance matrix for subjects:
Sigma_u <- diag(tau_u, 3, 3) %*% Cor_u %*% diag(tau_u, 3, 3)
# Create the correlated adjustments:
u <- mvrnorm(n = N_subj, c(0, 0, 0), Sigma_u)
# Check whether they are correctly correlated
# (There will be some random variation,
# but if you increase the number of observations,
# you'll be able to obtain more exact values below):
cor(u)
```

```
##           [,1]      [,2]      [,3]
## [1,] 1.0000000 0.5143151 0.3139776
## [2,] 0.5143151 1.0000000 0.2452616
## [3,] 0.3139776 0.2452616 1.0000000
```

```
# Check the SDs:
sd(u[, 1]); sd(u[, 2]); sd(u[, 3])
```

```
## [1] 0.2210712
```

```
## [1] 0.00586247
```

```
## [1] 0.2945844
```

```
# Create the simulated data:
df_dots_simdata <- stimuli %>%
  mutate(z = rbern(N, prob = plogis(alpha_task + x2 * beta_task)),
         rt = ifelse(z,
```

```
        rlnorm(N, meanlog = alpha + u[subj, 1] +
                       (beta + u[subj, 2]) * x,
               sdlog = sigma),
        rlnorm(N, meanlog = gamma + u[subj, 3],
               sdlog = sigma2)),
    acc = ifelse(z, rbern(n = N, p_correct),
         rbern(n = N, 0.5)),
    diff = if_else(x == 0.5, "hard", "easy"),
    emphasis = ifelse(x2 == 0.5, "accuracy", "speed"))
```

Verify that the distribution of the simulated response times conditional on the simulated accuracy and the experimental manipulations make sense; see the Figure below.

```
ggplot(df_dots_simdata, aes(x = factor(acc), y = rt)) +
  geom_point(position = position_jitter(width = 0.4,
                                        height = 0), alpha = 0.5) +
  facet_wrap(diff ~ emphasis) +
  xlab("Accuracy") +
  ylab("Response time")
```

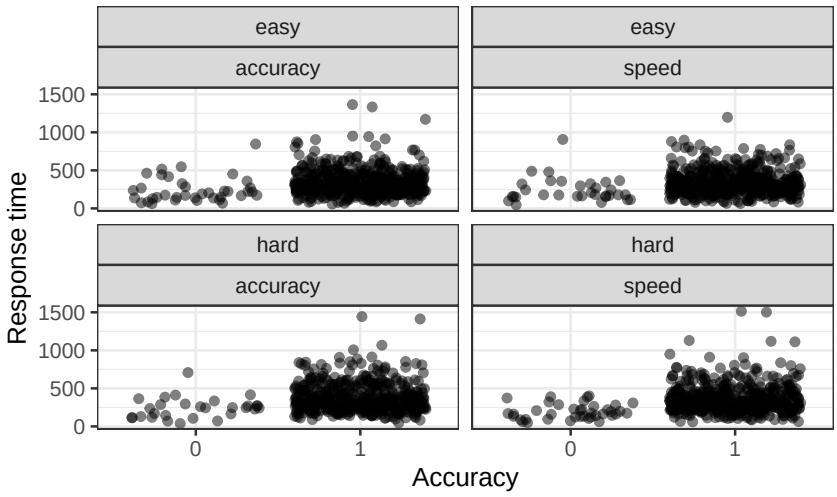

Figure 12: The distribution of response times conditional on the simulated accuracy and the experimental manipulations for the simulated hierarchical data that follows the fast-guess model.

We implement the model in Stan as follows in `mixture_h.stan`. The hierarchical extension uses the Cholesky factorization for the group-level effects.

```
data {
  int<lower = 1> N;
  vector[N] x;
  vector[N] rt;
  array[N] int acc;
  vector[N] x2;
  int<lower = 1> N_subj;
  array[N] int<lower = 1, upper = N_subj> subj;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
  real<upper = alpha> gamma;
  real<lower = 0> sigma2;
  real<lower = 0, upper = 1> p_correct;
  real<lower = 0, upper = 1> p_btask;
  real beta_task;
  vector<lower = 0>[3]  tau_u;
  matrix[3, N_subj] z_u;
  cholesky_factor_corr[3] L_u;
}
transformed parameters {
  matrix[N_subj, 3] u;
  u = (diag_pre_multiply(tau_u, L_u) * z_u)';
}
model {
  target += normal_lpdf(alpha | 6, 1);
  target += normal_lpdf(beta | 0, 0.1);
  target += normal_lpdf(sigma | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(gamma | 6, 1) -
    normal_lcdf(alpha | 6, 1);
  target += normal_lpdf(sigma2 | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(tau_u | 0, 0.5)
    - 3* normal_lccdf(0 | 0, 0.5);
  target += normal_lpdf(beta_task | 0, 1);
```

```
  target += beta_lpdf(p_correct | 995, 5);
  target += beta_lpdf(p_btask | 8, 2);
  target += lkj_corr_cholesky_lpdf(L_u | 2);
  target += std_normal_lpdf(to_vector(z_u));
  for(n in 1:N){
    real lodds_task = logit(p_btask) + x2[n] * beta_task;
    target +=
      log_sum_exp(log_inv_logit(lodds_task) +
          lognormal_lpdf(rt[n] | alpha + u[subj[n], 1] +
                         x[n] * (beta + u[subj[n], 2]), sigma) +
          bernoulli_lpmf(acc[n] | p_correct),
          log1m_inv_logit(lodds_task) +
          lognormal_lpdf(rt[n] | gamma + u[subj[n], 3], sigma2) +
          bernoulli_lpmf(acc[n] |0.5));
  }
}
generated quantities {
  corr_matrix[3] rho_u = L_u * L_u';
}
```

Save the model code and fit it to the simulated data:

```
ls_dots_simdata <- list(N = N,
                        rt = df_dots_simdata$rt,
                        x = df_dots_simdata$x,
                        x2 = df_dots_simdata$x2,
                        acc = df_dots_simdata$acc,
                        subj = df_dots_simdata$subj,
                        N_subj = N_subj)
mixture_h <- system.file("stan_models",
                         "mixture_h.stan",
                         package = "bcogsci")
fit_mix_h <- stan(file = mixture_h,
                  data = ls_dots_simdata,
                  iter = 3000,
                  control = list(adapt_delta = 0.9))
```

Print the posterior summary:

```
print(fit_mix_h, pars = c("alpha", "beta", "sigma", "gamma",
                          "sigma2", "p_correct", "p_btask",
```

```
                        "beta_task", "tau_u",
                        "rho_u[1,2]", "rho_u[1,3]",
                        "rho_u[2,3]"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=3000; warmup=1500; thin=1;
## post-warmup draws per chain=1500, total post-warmup draws=6000.
##
##             mean se_mean   sd  2.5%   25%   50%  75% 97.5% n_eff Rhat
## alpha       5.81    0.00 0.05  5.71  5.78  5.81 5.84  5.91   870 1.01
## beta        0.05    0.00 0.02  0.01  0.04  0.05 0.07  0.09  9053 1.00
## sigma       0.40    0.00 0.01  0.38  0.39  0.40 0.40  0.41 10503 1.00
## gamma       5.20    0.00 0.09  5.03  5.15  5.20 5.26  5.38  3129 1.00
## sigma2      0.52    0.00 0.03  0.45  0.49  0.52 0.54  0.59  5345 1.00
## p_correct   0.99    0.00 0.00  0.99  0.99  0.99 1.00  1.00  7123 1.00
## p_btask     0.87    0.00 0.01  0.85  0.87  0.87 0.88  0.90  8973 1.00
## beta_task   0.24    0.00 0.18 -0.11  0.12  0.24 0.37  0.60 12358 1.00
## tau_u[1]    0.23    0.00 0.04  0.16  0.20  0.22 0.25  0.32  1561 1.00
## tau_u[2]    0.03    0.00 0.02  0.00  0.01  0.03 0.05  0.09  3418 1.00
## tau_u[3]    0.32    0.00 0.08  0.18  0.26  0.31 0.37  0.50  2614 1.00
## rho_u[1,2]  0.05    0.00 0.38 -0.69 -0.23  0.06 0.34  0.75  9248 1.00
## rho_u[1,3]  0.37    0.00 0.22 -0.10  0.23  0.39 0.53  0.74  4407 1.00
## rho_u[2,3] -0.01    0.01 0.38 -0.72 -0.30 -0.02 0.27  0.71   906 1.00
##
## Samples were drawn using NUTS(diag_e) at Tue Mar 31 16:07:57 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

We see that we can fit the hierarchical extension of our model to simulated data. Next, we'll evaluate whether we can recover the true point values of the parameters.

4.1 Recovery of the parameters

By "recovering" the true values of the parameters, we mean that the true point values are somewhere inside the bulk of the posterior distribution of the model.

In the Figure below, we use `mcmc_recover_hist()` to compare the posterior distributions of the relevant param-

eters of the model with their true point values.

```
df_fit_mix_h <- fit_mix_h %>% as.data.frame() %>%
  select(c("alpha", "beta", "sigma", "gamma", "sigma2",
           "p_correct","p_btask", "beta_task", "tau_u[1]",
           "tau_u[2]", "tau_u[3]", "rho_u[1,2]", "rho_u[1,3]",
           "rho_u[2,3]"))
true_values <- c(alpha, beta, sigma, gamma, sigma2,
                 p_correct, p_btask, beta_task, tau_u[1],
                 tau_u[2], tau_u[3], rep(rho,3))
mcmc_recover_hist(df_fit_mix_h, true_values) +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1))
```

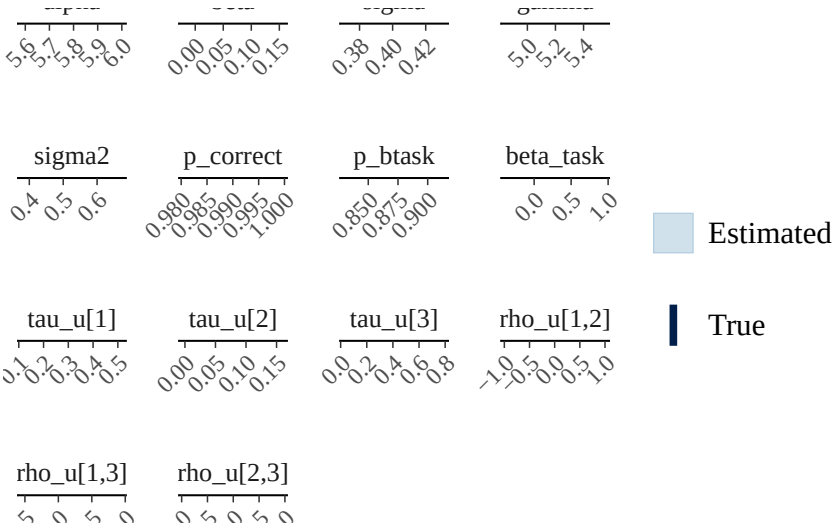

Figure 13: Posterior distributions of the main parameters of the mixture model together with their true point values.

The model seems to be underestimating the probability of subjects being correct (`p_correct`) and the amount of noise (`sigma`). However, the numerical differences are relatively small. We can be relatively certain that the model is not seriously misspecified. As mentioned in previous chapters, a more principled (and computationally demanding) approach uses simulation based calibration. An intermediate approach would be to re-run the simulation above a few times with a larger number of observations and/or with different true, data-generating parameters to see whether the estimates behave as expected.

4.2 *Fitting the model to real data*

After verifying that our model works as expected, we are ready to fit it to real data. We code the predictors x and x_2 as we did for the simulated data:

```
df_dots <- df_dots %>%
  mutate(x = if_else(diff == "easy", -0.5, 0.5),
         x2 = if_else(emphasis == "accuracy", 0.5, -0.5))
```

The main obstacle now is that fitting the entire data set takes around 12 hours! We'll sample 600 observations of each subject as follows:

```
df_dots_data_short <- df_dots %>%
  group_by(subj) %>%
  sample_n(600)

ls_dots_data_short <-
  list(N = nrow(df_dots_data_short),
       rt = df_dots_data_short$rt,
       x = df_dots_data_short$x,
       x2 = df_dots_data_short$x2,
       acc = df_dots_data_short$acc,
       subj = as.numeric(df_dots_data_short$subj),
       N_subj = length(unique(df_dots_data_short$subj)))
fit_mix_data <- stan(file = mixture_h,
                     data = ls_dots_data_short,
                     control = list(adapt_delta = .9,
                                    max_treedepth = 12))
```

```
## Warning: The largest R-hat is NA, indicating chains have not mixed.
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#r-hat
##
##  Warning: Bulk Effective Samples Size (ESS) is too low, indicating
## posterior means and medians may be unreliable. Running the chains for
## more iterations may help. See
## https://mc-stan.org/misc/warnings.html#bulk-ess
##
##  Warning: Tail Effective Samples Size (ESS) is too low, indicating
```

```
## posterior variances and tail quantiles may be unreliable. Running the
## chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#tail-ess
```

The model has not converged at all!

```
print(fit_mix_data,
      pars = c("alpha", "beta", "sigma", "gamma", "sigma2",
               "p_correct","p_btask", "beta_task", "tau_u",
               "rho_u[1,2]", "rho_u[1,3]", "rho_u[2,3]"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##            mean se_mean   sd  2.5%   25%  50%  75% 97.5% n_eff  Rhat
## alpha      6.36    0.03 0.05  6.29  6.33 6.35 6.39  6.46     3  1.62
## beta       0.09    0.02 0.03  0.06  0.07 0.07 0.10  0.15     2  3.10
## sigma      0.23    0.02 0.03  0.21  0.22 0.22 0.24  0.28     2 10.67
## gamma      6.24    0.06 0.09  6.06  6.18 6.28 6.31  6.35     2  2.68
## sigma2     0.36    0.07 0.09  0.19  0.35 0.41 0.42  0.42     2 20.14
## p_correct  0.98    0.02 0.03  0.92  0.97 0.99 1.00  1.00     2 13.96
## p_btask    0.74    0.06 0.08  0.68  0.69 0.70 0.75  0.91     2  9.08
## beta_task  1.83    1.26 1.79  0.67  0.77 0.84 1.83  5.31     2 12.05
## tau_u[1]   0.14    0.00 0.03  0.10  0.12 0.14 0.15  0.20    48  1.05
## tau_u[2]   0.03    0.01 0.01  0.01  0.02 0.03 0.04  0.06     5  1.30
## tau_u[3]   0.18    0.01 0.04  0.11  0.15 0.18 0.20  0.26     7  1.18
## rho_u[1,2] 0.38    0.00 0.26 -0.18  0.22 0.41 0.58  0.81  2795  1.00
## rho_u[1,3] 0.14    0.06 0.22 -0.31 -0.02 0.14 0.30  0.55    15  1.09
## rho_u[2,3] 0.28    0.01 0.27 -0.30  0.09 0.29 0.48  0.75   607  1.01
##
## Samples were drawn using NUTS(diag_e) at Tue Mar 31 16:32:06 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

The traceplots in the figure below show that the chains are not mixing at all. It seems that the posterior is multimodal, and there are at least two combinations of parameters that would fit the data equally well. (Rerunning the model might sometimes reveal three combinations.)

```
traceplot(fit_mix_data) +
  scale_x_continuous(breaks = NULL)
```

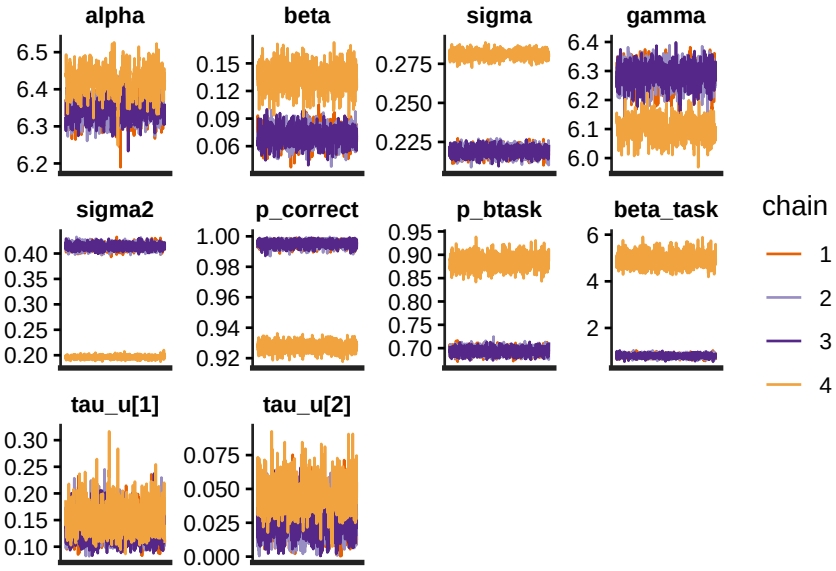

Figure 14: Traceplots from the hierarchical model fit to (a subset) of the real data. The traceplot shows clearly that the posterior has at least two modes, though rerunning the model might sometimes reveal three modes.

What should we do now? It can be a good idea to back off and simplify the model. Once the simplified model converges, we can think about adding further complexity. The verbal description of our model says that the accuracy in the task-engaged mode should be close to 100%. To simplify the model, we'll assume that it's exactly 100%. This entails the following:

$$p_{correct} = 1 \tag{29}$$

We adapt our Stan code in `mixture_h2.stan`, reflecting the assumption that `p_correct` has a fixed value; this parameter is now in a block called `transformed data`. There, we assign to `p_correct` the value of 1.

```
data {
  int<lower = 1> N;
  vector[N] x;
  vector[N] rt;
  array[N] int acc;
  vector[N] x2;
```

```
  int<lower = 1> N_subj;
  array[N] int<lower = 1, upper = N_subj> subj;
}
transformed data {
  real p_correct = 1;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
  real<upper = alpha> gamma; //guessing
  real<lower = 0> sigma2;
  real<lower = 0, upper = 1> p_btask;
  real beta_task;
  vector<lower = 0>[3]  tau_u;
  matrix[3, N_subj] z_u;
  cholesky_factor_corr[3] L_u;
}
transformed parameters {
  matrix[N_subj, 3] u;
  u = (diag_pre_multiply(tau_u, L_u) * z_u)';
}
model {
  target += normal_lpdf(alpha | 6, 1);
  target += normal_lpdf(beta | 0, 0.1);
  target += normal_lpdf(sigma | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(gamma | 6, 1) -
    normal_lcdf(alpha | 6, 1);
  target += normal_lpdf(sigma2 | 0.5, 0.2)
    - normal_lccdf(0 | 0.5, 0.2);
  target += normal_lpdf(tau_u | 0, 0.5)
    - 3* normal_lccdf(0 | 0, 0.5);
  target += normal_lpdf(beta_task | 0, 1);
  target += beta_lpdf(p_btask | 8, 2);
  target += lkj_corr_cholesky_lpdf(L_u | 2);
  target += std_normal_lpdf(to_vector(z_u));
  for(n in 1:N){
```

```
    real lodds_task = logit(p_btask) + x2[n] * beta_task;
    target +=
      log_sum_exp(log_inv_logit(lodds_task) +
          lognormal_lpdf(rt[n] | alpha + u[subj[n], 1] +
                         x[n] * (beta + u[subj[n], 2]), sigma) +
          bernoulli_lpmf(acc[n] | p_correct),
          log1m_inv_logit(lodds_task) +
          lognormal_lpdf(rt[n] | gamma + u[subj[n], 3], sigma2) +
          bernoulli_lpmf(acc[n] |0.5));
  }
}
generated quantities {
  corr_matrix[3] rho_u = L_u * L_u';
}
```

Fit the model again to the same data:

```
fit_mixs_data <- stan(file = mixture_h2,
                      data = ls_dots_data_short,
                      control = list(adapt_delta = .99,
                                     max_treedepth = 12))
```

The model has now converged:

```
print(fit_mixs_data,
      pars = c("alpha", "beta", "sigma", "gamma", "sigma2",
               "p_btask", "beta_task", "tau_u",
               "rho_u[1,2]", "rho_u[1,3]", "rho_u[2,3]"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##            mean se_mean   sd  2.5%   25%  50%  75% 97.5% n_eff Rhat
## alpha      6.34    0.00 0.03  6.28  6.32 6.34 6.36  6.40   607 1.01
## beta       0.07    0.00 0.01  0.05  0.06 0.07 0.08  0.09  2566 1.00
## sigma      0.22    0.00 0.00  0.21  0.22 0.22 0.22  0.22  5913 1.00
## gamma      6.29    0.00 0.04  6.22  6.27 6.29 6.31  6.36  1359 1.00
## sigma2     0.41    0.00 0.01  0.40  0.41 0.41 0.42  0.42  6947 1.00
## p_btask    0.69    0.00 0.01  0.67  0.68 0.69 0.69  0.70  6483 1.00
## beta_task  0.79    0.00 0.07  0.65  0.74 0.79 0.83  0.92  6094 1.00
```

```
## tau_u[1]   0.13    0.00 0.02  0.10  0.12 0.13 0.15  0.19  1026 1.00
## tau_u[2]   0.03    0.00 0.01  0.01  0.02 0.03 0.03  0.05  1094 1.00
## tau_u[3]   0.19    0.00 0.04  0.13  0.16 0.18 0.21  0.27  1571 1.00
## rho_u[1,2] 0.39    0.00 0.28 -0.22  0.21 0.42 0.60  0.83  3555 1.00
## rho_u[1,3] 0.10    0.00 0.21 -0.31 -0.04 0.11 0.25  0.49  2025 1.00
## rho_u[2,3] 0.28    0.01 0.29 -0.34  0.10 0.29 0.49  0.77   497 1.00
##
## Samples were drawn using NUTS(diag_e) at Tue Mar 31 17:17:59 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
traceplot(fit_mixs_data) +
  scale_x_continuous(breaks = NULL)
```

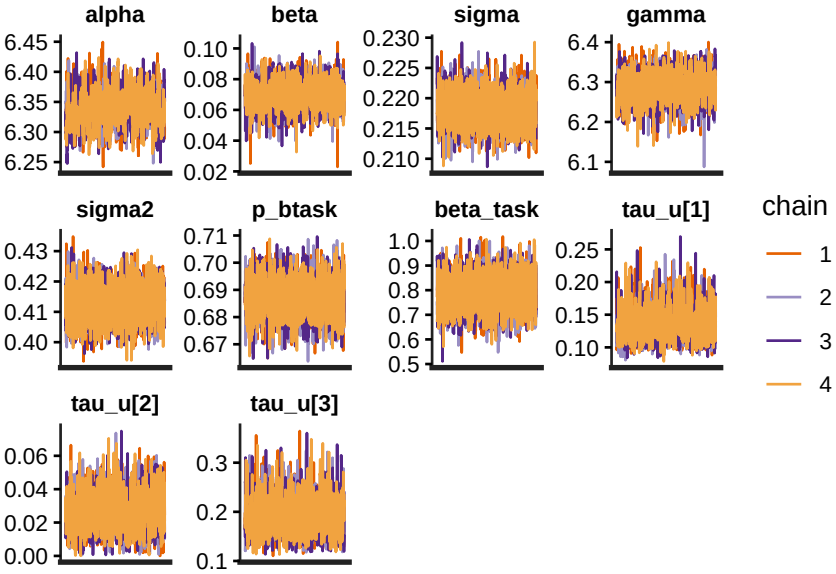

Figure 15: Traceplots from the simplified hierarchical model. The traceplot shows that chains are mixing well.

What can we say about the fit of the model now?

Under the assumptions that we have made (e.g., that there are two processing modes, response times are affected by the difficulty of the task in the task-engaged mode, accuracy is not affected by the difficulty of the task and is perfect at the task-engaged mode, etc.), we can look at the parameters and conclude the following:

- The instructions seemed to have a strong effect on the processing mode of the subjects (`beta_task` is relatively

high), and in the expected direction (emphasis in accuracy led to a higher probability to be in the task engaged mode).
- The guessing mode seemed to be much noisier than the task-engaged mode (compare `sigma` with `sigma2`).

If we want to know whether our model achieves descriptive adequacy, we need to look at the posterior predictive distributions of the model. However, by using posterior predictive checks, we won't be able to conclude that our model is not overfitting. Our success in fitting the fast-guess model to real data does not entail that the model is a good account of the data. It just means that it's flexible enough to fit the data. One further step would be to test whether each parameter can be selectively influenced by specific experimental manipulations as theoretically predicted. Another step could be to develop a competing model and then compare the performance of the models using Bayes factors or cross-validation.

4.3 Posterior predictive checks

For the posterior predictive checks, we can write the generated quantities block in a new file; in the `bcogsci` package, this code is in the file `mixture_h2_gen.stan`. The advantage of creating a new file is that we can generate as many observations as needed *after estimating the parameters*. There is no model block in the following Stan program and the parameters together with all the transformed parameters appear in the `parameters` block. We use the `gqs()` function in the `rstan` library, which allows us to use the posterior draws from a previously fitted model to generate posterior predicted data.

```
data {
  int<lower = 1> N;
  vector[N] x;
  vector[N] rt;
  array[N] int acc;
  vector[N] x2;
```

```
  int<lower = 1> N_subj;
  array[N] int<lower = 1, upper = N_subj> subj;
}
transformed data{
  real p_correct = 1;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
  real<upper = alpha> gamma; //guessing
  real<lower = 0> sigma2;
  real<lower = 0, upper = 1> p_btask;
  real beta_task;
  vector<lower = 0>[3]  tau_u;
  matrix[3, N_subj] z_u;
  cholesky_factor_corr[3] L_u;
  matrix[N_subj, 3] u;
}
generated quantities {
  array[N] real rt_pred;
  array[N] real acc_pred;
  array[N] int z;
  for(n in 1:N){
    real lodds_task = logit(p_btask) + x2[n] * beta_task;
    z[n] = bernoulli_rng(inv_logit(lodds_task));
    if(z[n]==1){
      rt_pred[n] = lognormal_rng(alpha + u[subj[n],1] +
                                 x[n] * (beta + u[subj[n], 2]), sigma);
      acc_pred[n] = bernoulli_rng(p_correct);
    } else{
      rt_pred[n] = lognormal_rng(gamma + u[subj[n], 3], sigma2);
      acc_pred[n] = bernoulli_rng(0.5);
    }
  }
}
```

Generate responses from 500 simulated experiments as follows:

```
mixture_h2_gen <- system.file("stan_models",
                              "mixture_h2_gen.stan",
                              package = "bcogsci")
gen_model <- stan_model(mixture_h2_gen)
# The argument of the matrix `drop` needs to be set to FALSE,
# otherwise R will simplify the matrix into a vector.
# The two commas in the line below are not a mistake!
draws_par <- as.matrix(fit_mixs_data)[1:500, , drop = FALSE]
gen_mix_data <- gqs(gen_model,
                    data = ls_dots_data_short,
                    draws = draws_par)
```

First, take a look at the general distribution of response times generated by the posterior predictive model and by our real data.

```
rt_pred <- rstan::extract(gen_mix_data)$rt_pred
ppc_dens_overlay(y = ls_dots_data_short$rt, yrep = rt_pred[1:100,]) +
  coord_cartesian(xlim = c(0, 2000))
```

We see that the distribution of the observed response times is narrower than the predictive distribution. We are generating response times that are more spread out than the real data.

Next, examine the effect of the experimental manipulation : The posterior predictive check reveals that the model underestimates the observed effect of the experimental manipulation: the observed difference between response times is well outside the bulk of the predictive distribution.

```
meanrt_diff <- function(rt){
  mean(rt[ls_dots_data_short$x == 0.5]) -
    mean(rt[ls_dots_data_short$x == -0.5])
}
ppc_stat(ls_dots_data_short$rt,
         yrep = rt_pred,
         stat = meanrt_diff)
```

Another important posterior predictive check includes comparing the fit of the model using a quantile probability plot, which is presented in the next chapter.

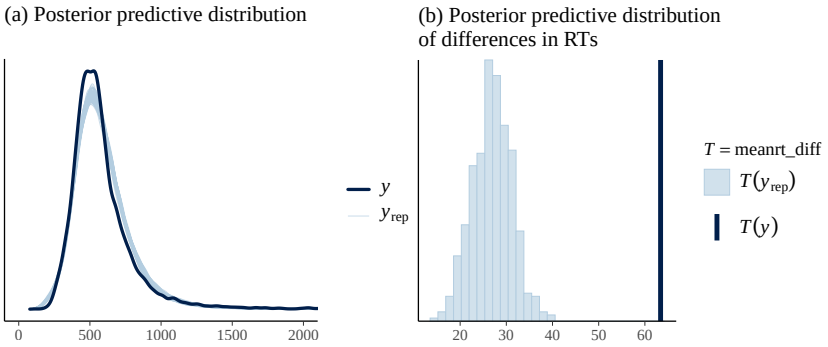

Figure 16: (a) Posterior predictive distribution of the hierarchical fast-guess model compared to observed response times. (b) Posterior predictive distribution of the response time difference due to experimental manipulation, using the same model; the vertical bar indicates the observed difference in the data.

We also look at some instances of the predictive distribution. Figure @ref(fig:postppdobs) shows a simulated data set in black overlaid onto the real observations in gray. As we noticed in Figure @ref(fig:postpmix-postpmanip)(a), the model is predicting less variability than what we find in the data, especially when the emphasis is on accuracy.

```
acc_pred <- rstan::extract(gen_mix_data)$acc_pred
df_dots_pred <-
  tibble(rt = ls_dots_data_short$rt,
         acc = ls_dots_data_short$acc,
         difficulty = ifelse(ls_dots_data_short$x == 0.5,
                             "hard", "easy"),
         emphasis = ifelse(ls_dots_data_short$x2 == -0.5,
                           "speed", "accuracy"),
         acc_pred1 = acc_pred[1,],
         rt_pred1 = rt_pred[1,])
```

If we would like to compare this model with a competing one using cross-validation, we would need to calculate the point-wise log-likelihood in the generated block:

```
generated quantities {
  array[N] real log_lik;
  for(n in 1:N){
    real lodds_task = logit(p_btask) + x2[n] * beta_task;
    log_lik[n] += log_sum_exp(log_inv_logit(lodds_task) +
                              lognormal_lpdf(rt[n] | alpha + u[subj[n], 1] +
                                             x[n] * (beta + u[subj[n], 2]), sigma) +
                              bernoulli_lpmf(acc[n] | p_correct),
```

```
                                log1m_inv_logit(lodds_task) +
                                  lognormal_lpdf(rt[n] | gamma + u[subj[n], 3], sigma2) +
                                  bernoulli_lpmf(acc[n] | 0.5));
  }
}
```

It is important to bear in mind that we can only compare models on the same dependent variable(s). That is, we would need to compare this model with another one fit to the same dependent variables and also in the same scale: accuracy (`0` or `1`) and response time in milliseconds. This means that, for example, we cannot compare our fast-guess model with an accuracy-only model. It also means that to compare our fast-guess model with a model based on left/right choices (known as stimulus coding, see section @ref(sec-acccoding)) and response times, we would need to reparameterize one of the two models; see the online exercise @ref(exr:lnldt) for chapter @ref(ch-lognormalrace).

To conclude, the fast-guess model shows a relatively decent fit to the data and is able to account for the speed-accuracy trade-off. The model shows some inaccuracies that could lead to its revision and improvement. To what extent the inaccuracies are acceptable or not depends on (i) the empirical finding that we want to account for (for example, we can already assume that the model will struggle to fit data sets that show slow errors); and (ii) its comparison with a competing account.

4.4 Summary

In this chapter, we learned how to fit increasingly complex two-component mixture models using Stan, starting with a simple model and ending with a fully hierarchical model. We saw how to ensure model convergence via prior constraints, and how to evaluate model fit using the usual prior and posterior predictive checks, and to investigate parameter recovery. Such mixture models are notoriously difficult to fit, but they have a lot of potential

in cognitive science applications, especially in developing computational models of different kinds of cognitive processes.

4.5 Further reading

See the references in the chapter in the textbook.